

Complete Project Guide

EU Power-Market Data-as-a-Service & Explainable Load Forecasting

Everything this system does, how to deploy it, how to use it, which data source powers which feature, and exactly where each API key goes. Written to take you from a fresh clone to a live, credentialed deployment serving the Italy Power Watch dashboard and the developer API.

Prices · ENTSO-E A44

Load · ENTSO-E A65

Flows · ENTSO-E A11

Weather · MET Norway / GFS / DWD / ECMWF

GB demand · NESO

Forecast · LightGBM + SHAP

Contents

- 01 What the project is** — the problem, the product, the parts

- 02 How it works end-to-end** — architecture & data flow

- 03 Data sources & what each is used for** — the full data map

- 04 API keys: which key, which file, which line** — the credential map

- 05 Putting it live** — demo mode, then real data, then production

- 06 Using the dashboard** — Italy Power Watch, panel by panel

- 07 Using the API** — every endpoint with examples

- 08 The forecasting model** — how it works & how it proves itself

- 09 Excel & Power BI** — the analyst workflow

- 10 Security & credential rotation** — read this before deploying

- 11 Troubleshooting & FAQ** — common issues

1 · What the project is

NRG-Flux is a developer-first data provider for European power markets. It takes the authoritative-but-awkward **ENTSO-E Transparency Platform** and turns it into a clean, reliable product: a REST API, a live dashboard, Excel/Power BI connectors, and a premium **explainable load-forecasting** layer. The first market is Italy; Great Britain is included as the first expansion via NESO.

The problem it solves

ENTSO-E is the legal source for EU electricity data, but using it directly means fighting XML documents, cryptic EIC area codes, request-window limits, data revisions, and daylight-saving edge cases where a market day can start at 22:00 or 23:00 UTC. NRG-Flux absorbs all of that and hands back clean JSON with units, UTC + market-day timestamps, lineage, and quality flags on every row.

What it produces

Deliverable	What it is
Core Market Data API	Day-ahead prices, actual load, TSO load forecast, cross-border physical flows, outages, EIC resolver.
Explainable forecasting	D+1...D+7 hourly load forecast with p10/p50/p90 uncertainty bands and per-hour SHAP driver attributions (why demand moves).
Italy Power Watch	A public-facing dashboard: price ticker, zonal prices, animated grid schematic, load-vs-forecast, SHAP panel, flows, outages.
Status & quality layer	Source freshness, fill-rate, and data-quality events — the "can I trust this" surface.
Excel / Power BI	Power Query recipes so analysts can refresh live data without code.

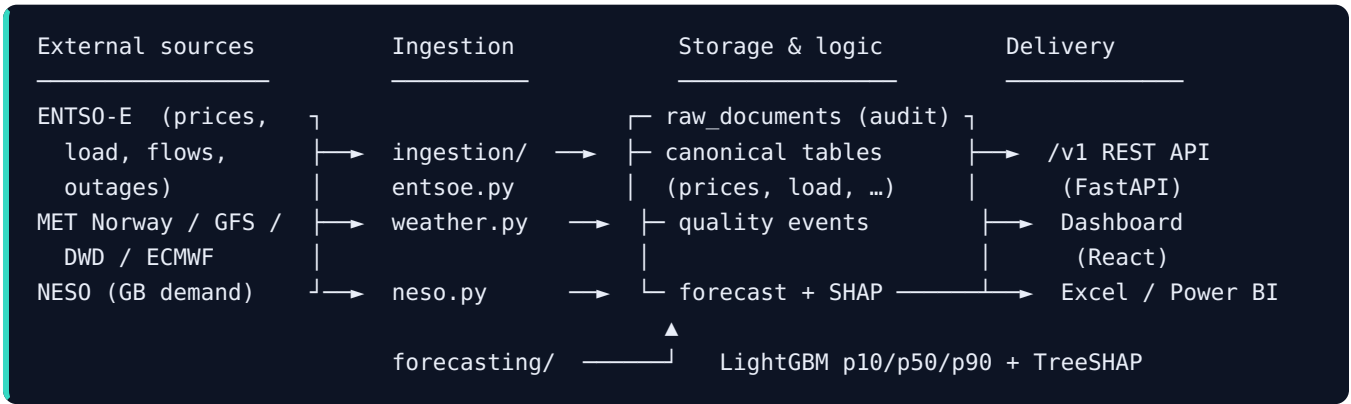
The repository at a glance

```
nrg-flux/
├── backend/                # FastAPI service + forecasting + ingestion
│   ├── app/
│   │   ├── api/           # /v1 routers: market, forecast, meta
│   │   ├── ingestion/     # entsoe.py · weather.py · neso.py
│   │   ├── parsers/       # ENTSO-E XML → clean rows
│   │   ├── forecasting/   # features · LightGBM model · SHAP
│   │   ├── quality/       # gap / stale / outlier checks
│   │   ├── demo/          # synthetic data generator
│   │   ├── db/            # canonical schema (SQLite → Postgres)
│   │   ├── utils/         # UTC/DST time + EIC resolver
│   │   └── config.py       # ← all credentials load here (from .env)
│   ├── scripts/           # seed_demo · train_forecast · backfill_*
│   └── tests/             # DST golden tests, parsers, API smoke
├── frontend/              # Vite + React + ECharts dashboard
├── docs/                  # excel-powerbi · weather · preview.html
├── docker-compose.yml     # one-command full stack
└── .env.example           # ← copy to backend/.env and fill in
```

2 · How it works end-to-end

The design is deliberately boring and modular — it wins on data quality and explainability, not exotic infrastructure.

Data flow



The two modes

Mode	What happens	Set by
Demo (default)	A synthetic-but-realistic Italian market generator fills the database, so the entire stack — dashboard, API, forecast, SHAP — works with zero credentials . Ideal for evaluating and for the sales demo.	DEMO_MODE=true
Live	Real ENTSO-E (and optionally NESO) data is backfilled into the same tables; forecasts use real weather. Identical API and dashboard — just real numbers.	DEMO_MODE=false

KEY IDEA

Demo and live use the **same database tables and the same API**. Switching modes doesn't change the interface — it only changes where the numbers come from. You can build, demo, and sell on demo data, then flip to live the moment your ENTSO-E access is approved.

Technology

Backend	Python 3.12, FastAPI, SQLAlchemy, Pydantic. SQLite by default (one file), Postgres/TimescaleDB-ready.
Forecasting	LightGBM quantile gradient boosting, native TreeSHAP for explanations, pandas/ numpy features.
Frontend	React 18, Vite, ECharts 5. Served by nginx in production.
Deployment	Docker Compose (API + web). One volume holds the SQLite DB and trained models.

3 · Data sources & what each is used for

This is the heart of the system — the "which data does what" map.

Source	Used for	Key needed?	Notes
ENTSO-E Transparency Platform	The core market data: day-ahead prices (doc A44), actual load and TSO load forecast (A65), cross-border physical flows (A11), outages (A77/A80). Also the training target for the forecast model.	Yes — required for live data	Register, then request REST access by email. Approval can take up to 3 business days; until then calls return 401.
MET Norway Locationforecast	The default forward weather forecast (temperature) feeding D+1...D+7 load features, sampled at each bidding-zone centroid.	No key — just a User-Agent	Their terms require an identifying User-Agent with contact info. That's the only credential.
NOAA GFS on AWS Open Data	Alternative / blendable weather forecast source (global model, 4 runs/day).	No key (anonymous S3)	GRIB2 files; decoding to zone temperature needs the optional cfgrib extra.
DWD ICON, opendata.dwd.de	Alternative weather forecast, strong over central Europe.	No key	Keyless HTTPS directory of GRIB files.
ECMWF Open Data	Higher-skill alternative weather forecast at these horizons.	No key for open data	A key is only for licensed/archive datasets. Install the ecmwf-opendata extra to use it.
NESO GB Data Portal (CKAN)	Great Britain national demand (half-hourly, normalized to hourly UTC) — the first coverage expansion beyond Italy.	Optional	Open dataset needs no key; a key only raises rate limits.

IMPORTANT REALITY CHECK ON YOUR KEYS

Most of the weather keys you hold **are not actually needed** for the endpoints this project uses: MET Norway authenticates by User-Agent, NOAA GFS on AWS is anonymous, DWD is keyless, and ECMWF open data is free without a key. The one credential that genuinely gates live data is the **ENTSO-E token** — and that requires ENTSO-E to approve your REST access first.

How weather becomes a forecast feature

For each bidding zone the system takes a temperature forecast (MET Norway by default) at the zone centroid, derives heating/cooling degree values and a temperature anomaly, and combines them with calendar features (hour, weekday, holiday, DST) and load-lag features (yesterday and last-week same-hour, rolling averages). The upgrade path — ERA5 history + population-weighted zone temperature using the Eurostat GISCO grid — is documented in [docs/weather.md](#).

4 · API keys: which key, which file, which line

GOLDEN RULE

Every credential lives in **one file only**: `backend/.env`. That file is git-ignored and never committed. No key is ever hardcoded in source. You create it by copying the template: `cp .env.example backend/.env`.

Step 1 — create your env file

```
# from the repo root
cp .env.example backend/.env
# then open backend/.env in an editor and fill in the values below
```

Step 2 — fill in each variable

Every row below is a line in `backend/.env`. Left = the variable name, right = what to put and where the value comes from.

Variable in <code>backend/.env</code>	What to set it to
ENTSOE_API_TOKEN	Your ENTSO-E Web API token. From transparency.entsoe.eu → <i>My Account Settings</i> → <i>Web API Security Token</i> (after they approve REST access by email). Required for live market data.
DEMO_MODE	<code>true</code> to run on synthetic data; <code>false</code> once you've backfilled real ENTSO-E data.
API_KEYS	Comma-separated list of keys <i>your</i> API will accept from callers, e.g. <code>demo-key,client-acme</code> . Sent by clients in the <code>X-API-Key</code> header.
DATABASE_URL	Leave as the SQLite default for a single-box deploy, or a Postgres URL for production.
METNO_USER_AGENT	An identifying string with your contact, e.g. <code>nrg-flux/1.0 you@yourdomain.com</code> . This is MET Norway's required "credential."
METNO_CLIENT_ID METNO_API_KEY	Optional — only for MET Norway's authenticated products. Leave blank for the forecast endpoint used here.
ECMWF_API_KEY	Optional — only if you enable licensed ECMWF datasets. Open-data forecasts need nothing here.
NOAA_API_TOKEN	Optional — only for NOAA NCEI/CDO climate web services. GFS-on-AWS ignores it.
NESO_API_KEY	Optional — raises NESO rate limits. The GB demand dataset works without it.
WEBHOOK_SIGNING_SECRET	A long random string used to HMAC-sign webhook payloads.

What a filled `backend/.env` looks like

```
# runtime
ENVIRONMENT=development
DEMO_MODE=true           # flip to false after a live backfill
DATABASE_URL=sqlite:///./nrgflux.db
```

```
API_KEYS=demo-key          # keys YOUR api accepts (X-API-Key header)

# ENTSO-E – the one required credential for live data
ENTSOE_API_TOKEN=4f1d2563-xxxx-xxxx-xxxx-xxxxxxxxxxxxx

# MET Norway – just an identifying User-Agent, no key
METNO_USER_AGENT=nrg-flux/1.0 you@yourdomain.com

# NESO (GB) – optional, only for higher rate limits
NESO_API_KEY=

# these are optional / usually left blank (see notes above)
ECMWF_API_KEY=
NOAA_API_TOKEN=
WEBHOOK_SIGNING_SECRET=change-me-to-something-random
```

How the keys are read (so you can trust nothing is hardcoded)

All variables are loaded in exactly one place — `backend/app/config.py` — via Pydantic settings that read from `backend/.env`. Ingestion code asks that config object for a value; it never contains a literal key. If a required token is missing, the code logs a warning and (in demo mode) simply serves synthetic data instead.

FOR DOCKER

Compose automatically loads `backend/.env` into the `api` container. You don't set keys anywhere else — same file, whether you run locally or in Docker.

5 · Putting it live

Three stages: prove it on demo data, switch to real data, then harden for production.

Stage A — run it now (demo data, zero credentials)

Option 1: Docker (recommended)

```
docker compose up --build
```

First start seeds demo data and trains the forecaster automatically (~1 minute). Then open:

URL	What
http://localhost:8080	Italy Power Watch dashboard
http://localhost:8080/docs	Interactive API documentation (Swagger)
http://localhost:8080/v1/status	Source freshness / trust surface

Option 2: local dev (two terminals)

```
# terminal 1 – backend
cd backend
pip install -r requirements.txt
python scripts/seed_demo.py          # fill DB with demo market data
python scripts/train_forecast.py     # train + backtest + issue forecast
uvicorn app.main:app --reload --port 8000

# terminal 2 – frontend
cd frontend
npm install
npm run dev                          # http://localhost:5173
```

Stage B — switch to real data

1. Get ENTSO-E access: register at transparency.entsoe.eu, then email them to request Restful API access. Once approved, copy your token.
2. Put the token in `backend/.env` as `ENTSOE_API_TOKEN=...`.
3. Backfill and switch modes:

```
cd backend
python scripts/backfill_entsoe.py --days 30  # pull real Italian data
# set DEMO_MODE=false in backend/.env
python scripts/train_forecast.py             # retrain on real load
# restart the API
```

Optional — add Great Britain:

```
python scripts/backfill_neso.py --days 30    # GB national demand from NES0
# now queryable at /v1/load/actual?area=10YGB-----A
```

YOUR ENTSO-E STATUS RIGHT NOW

Your account is set up but REST API access was still pending approval (their email can take up to 3 business days). Until that arrives, live ENTSO-E calls will return **401 Unauthorized** even with the token in place — this is expected. Keep running in demo mode until then; nothing else needs to change when it activates.

Stage C — production hardening

Do this	Why / how
Lock down CORS	Set <code>CORS_ORIGINS</code> to your real domain(s), not <code>*</code> .
Real API keys	Replace <code>demo-key</code> in <code>API_KEYS</code> with per-customer keys.
Move to Postgres	Uncomment the TimescaleDB block in <code>docker-compose.yml</code> and set <code>DATABASE_URL</code> accordingly.
Schedule ingestion	Run the backfill scripts on a scheduler (cron / Prefect / Airflow) so data stays fresh; retrain weekly.
HTTPS + monitoring	Terminate TLS at a reverse proxy; watch <code>/v1/status</code> freshness and <code>/v1/quality/events</code> .

6 · Using the dashboard

Italy Power Watch is the public proof asset — and your fastest demo. Every panel is fed by one aggregate call to `/v1/dashboard/italy`, refreshed every two minutes. A pill in the header shows whether you're on **DEMO DATA** or **LIVE · ENTSO-E**.

Panel	What it shows
Price ticker (top strip)	Latest day-ahead price per bidding zone with the 24-hour change arrow.
KPI row	Average zonal price, tomorrow's forecast peak load (p50 with band), the widest zone-to-zone price spread, and total unavailable capacity.
Day-ahead prices by zone	Multi-line chart of all six Italian zones over the last 72h plus the published day-ahead. Hover for exact values; click legend entries to isolate a zone.
Zonal grid schematic	A single-line diagram of the Italian zones with animated interconnector flow lines — line width scales with live import MW, and each zone shows its current price.
Load: actual vs forecast	Actual load, the NRG-Flux p50 forecast, the shaded p10–p90 uncertainty band, and the TSO's own day-ahead forecast (dashed) for direct comparison. The tag shows the model's backtest WAPE vs the naive baseline.
Why demand moves (SHAP)	For any forecast hour (drag the slider), the top drivers pushing demand up (red) or down (teal) in MW — temperature, day-of-week, load-lags, and so on.
Cross-border imports	Latest-hour physical flows on each interconnector as a ranked bar chart.
Largest unavailabilities	The biggest generation/grid outages by MW, flagged planned vs forced.

TRY IT OFFLINE

A fully self-contained preview of this exact interface ships at <docs/preview.html> — open it in any browser with no server running to explore the layout and interactions on demo data.

7 · Using the API

Every endpoint is under `/v1`. All data endpoints require the `X-Api-Key` header (any value in your `API_KEYS` list; `demo-key` works out of the box). Full interactive docs live at `/docs`.

Authentication

```
curl -H "X-Api-Key: demo-key" \
      "http://localhost:8000/v1/prices/dayahead?area=10Y1001A1001A73I"
```

Market data endpoints

Method	Path	Returns
GET	<code>/v1/prices/dayahead</code>	Day-ahead prices for an area over a window.
GET	<code>/v1/load/actual</code>	Actual load for an area.
GET	<code>/v1/load/forecast/tso</code>	The TSO's own day-ahead load forecast.
GET	<code>/v1/flows/physical</code>	Physical cross-border flow between two areas.
GET	<code>/v1/outages</code>	Generation/grid unavailabilities.
GET	<code>/v1/eic/resolve</code>	Resolve a name (e.g. "Sicily", "UK") to an EIC area code.

Forecasting endpoints

Method	Path	Returns
GET	<code>/v1/forecast/load</code>	Latest D+1...D+7 forecast with p10/p50/p90 + backtest summary.
GET	<code>/v1/forecast/explain</code>	SHAP driver breakdown per forecast hour.
POST	<code>/v1/forecast/whatif</code>	Scenario: shift the temperature path, see the load response.
GET	<code>/v1/forecast/modelcard</code>	Model card: training window, features, backtest, known weaknesses.

Platform endpoints

Method	Path	Returns
GET	<code>/v1/status</code>	Per-dataset freshness, row counts, lag. No key required.
GET	<code>/v1/quality/events</code>	Recent gap/stale/outlier/revision events.
POST	<code>/v1/webhooks</code>	Register a webhook for price/outage/forecast events.
GET	<code>/v1/dashboard/italy</code>	The aggregate feed powering the dashboard.

Example: a forecast with explanations

```
curl -H "X-API-Key: demo-key" \
  "http://localhost:8000/v1/forecast/load?horizon=24"
```

```
{
  "area_eic": "10YIT-GRTN-----B",
  "issued_at_utc": "2026-07-12T00:00:00Z",
  "forecast": [
    { "ts_utc": "2026-07-12T18:00:00Z",
      "mw_p10": 43800, "mw_p50": 45250, "mw_p90": 46850 }
  ],
  "backtest_summary": { "wape_model": 1.2, "wape_naive_weekly": 4.86 },
  "beats_naive_baseline": true
}
```

Italian bidding-zone EIC codes (handy reference)

Zone	EIC	Zone	EIC
Italy (national)	10YIT-GRTN-----B	South (SUD)	10Y1001A1001A788
North (NORD)	10Y1001A1001A73I	Sardinia (SARD)	10Y1001A1001A74G
Centre-North (CNOR)	10Y1001A1001A700	Sicily (SICI)	10Y1001A1001A75E
Centre-South (CSUD)	10Y1001A1001A71M	Great Britain	10YGB-----A

8 · The forecasting model

Forecasting is the premium module — it moves NRG-Flux from data delivery into decision support.

What it predicts

Hourly electricity load per area for horizons D+1 through D+7, as three quantiles: **p10** (low), **p50** (central estimate), and **p90** (high). The band between p10 and p90 is the uncertainty range you see shaded on the dashboard.

How it's built

Model	Three LightGBM gradient-boosting models (one per quantile), trained with the quantile objective.
Features	Calendar (hour, weekday, month, weekend, holiday, DST); weather (temperature, heating/cooling degrees, temperature anomaly); load history (yesterday same-hour, last-week same-hour, 24h and 7-day rolling means).
Explanations	LightGBM's native TreeSHAP gives exact per-feature MW contributions for every predicted hour — that's the "why demand moves" panel.
What-if	Shift the temperature path by $\pm N^{\circ}\text{C}$ and re-run to see the demand response.

How it proves itself (the honesty gate)

The plan's rule: *if the model can't beat a naive baseline, don't sell the forecast*. So `train_forecast.py` runs a rolling hold-out backtest against the naive "previous-week same-hour" baseline and writes a model card. Every forecast API response carries the backtest numbers and a `beats_naive_baseline` flag — the model can't hide underperformance.

DEMO BACKTEST RESULT

On the synthetic generator the model reaches **WAPE $\approx$ 1.2%** versus the naive baseline's **$\approx$ 4.9%**. These are demo-data numbers and will differ on live data — the point is the machinery reports the comparison honestly, per zone and horizon.

NOT FINANCIAL ADVICE

Forecasts are probabilistic decision-support outputs, not guaranteed trading outcomes, and the demo data is synthetic and not suitable for real trading decisions.

9 · Excel & Power BI

The go-to-market is Excel-first: analysts refresh live data without writing code. Both Excel (Power Query) and Power BI use the same approach against the NRG-Flux API. Full recipes are in [docs/excel-powerbi.md](#).

Excel (Power Query) — the short version

1. Data → Get Data → From Other Sources → Blank Query → Advanced Editor.
2. Paste an M query that calls the API with your key in the header:

```
let
    ApiKey = "demo-key",
    BaseUrl = "http://localhost:8000",
    Area = "10Y1001A1001A73I", // IT-North
    Source = Json.Document(Web.Contents(BaseUrl, [
        RelativePath = "v1/prices/dayahead",
        Query = [area = Area],
        Headers = [{"X-Api-Key" = ApiKey} ])),
    Table = Table.FromList(Source[series], Splitter.SplitByNothing())
in Table
```

Close & Load, then **Data** → **Refresh All** re-pulls live data (target refresh under 10 seconds). Repeat for load, flows, outages, and the forecast endpoint.

Power BI

Home → Get Data → Web → Advanced. Put the full URL in the URL parts and add an HTTP request header **X-Api-Key** with your key — or paste the same M query into a Blank Query. Schedule refresh on the service via a gateway if the API is inside your network.

10 · Security & credential rotation

ROTATE THE CREDENTIALS YOU SHARED

Several passwords and API keys were shared in plaintext during this project's setup (a Gmail password, ENTSO-E and NESO passwords, and various API keys). Treat all of them as **compromised** and rotate them before deploying anything publicly. This is the single most important action item in this document.

What to rotate, and where

Credential	Action
Gmail account password	Change it, and enable 2-factor authentication.
ENTSO-E password & token	Reset the password; regenerate the Web API token if it was exposed.
NESO password & API key	Reset the password; regenerate the API key in the NESO portal.
Any weather keys	Regenerate if exposed — though most aren't used by this project anyway.

Good hygiene going forward

- **Never commit** `.env` — it's already in `.gitignore`. Only `.env.example` (with blank placeholders) belongs in git.
- Don't paste real secrets into chats, tickets, or docs. Share them through a secrets manager.
- In production, inject secrets via your platform's secret store (Docker/K8s secrets, cloud secret manager) rather than a file on disk.
- Use separate, scoped `API_KEYS` per customer so you can revoke one without affecting others.
- Rotate the `WEBHOOK_SIGNING_SECRET` and any customer keys periodically.

VERIFIED

The shipped repository was scanned against every credential shared during setup — none are present in the code or the zip. All credentials load exclusively from your local `backend/.env`.

11 · Troubleshooting & FAQ

The dashboard says "can't reach the API"

The backend isn't running or is on a different port. Start it with `uvicorn app.main:app --port 8000` (or `docker compose up`). The dev frontend proxies `/v1` to port 8000.

Live ENTSO-E calls return 401

Either the token isn't set in `backend/.env`, or — more likely right now — ENTSO-E hasn't approved your REST API access yet (up to 3 business days after you email them). Stay in demo mode until the approval email arrives.

The forecast endpoint returns 404 "no_forecast"

No model has been trained/issued yet. Run `python scripts/train_forecast.py` (after `seed_demo.py` in demo mode, or after a backfill in live mode).

NESO backfill returns no records

NESO occasionally rotates its dataset resource IDs. Open the "Historic Demand Data" page on the NESO portal and update the IDs in `backend/app/ingestion/neso.py`.

Does anything actually need a paid key?

No. The whole live pipeline runs on free/open sources. The only *required* credential is the free ENTSO-E token; everything else is either keyless or optional-for-rate-limits.

How do I add another country/zone?

Add its EIC code to the resolver in `backend/app/utils/eic.py` (with a weather centroid), backfill its data, and train a forecaster for it with `--area <EIC>`. Great Britain is included as a worked example.

Where are the trained models stored?

In `backend/models/` (git-ignored). In Docker they persist on the `nrgflux-data` volume, so they survive restarts.

NRG-Flux — EU Power-Market Data-as-a-Service & Explainable Load Forecasting. Market data © ENTSO-E Transparency Platform. Weather: MET Norway, NOAA, DWD, ECMWF open data. GB demand: NESO. This guide covers the shipped codebase; see the in-repo `README.md`, `docs/weather.md`, and `docs/excel-powerbi.md` for deeper detail.